МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Львівська політехніка»
Інститут комп’ютерних наук та інформаційних технологій
Кафедра «Системи штучного інтелекту»



ЛАБОРАТОРНА РОБОТА №3

з дисципліни

«Методологія та процеси моделювання у штучному інтелекті»

на тему

Оптимізація гіперпараметрів, вкладена крос-валідація та дослідження витоку даних

Виконав:
студент групи КНСШ-13
Галич Максим

Викладач:
Косаревич Р.Я.

Львів — 2026

Мета роботи. Навчитися застосовувати методи оптимізації гіперпараметрів та крос-валідації; дослідити вплив вибору алгоритму пошуку на обчислювальну складність і якість моделей; опанувати методологію запобігання витоку даних та застосування вкладеної крос-валідації (Nested Cross-Validation).

Усі обчислення виконано мовою Python із бібліотеками scikit-learn [2], imbalanced-learn та Optuna [3]. Код продовжує пакет `mk1` з лабораторних робіт №1–2. Пошук гіперпараметрів описано в модулі `tuning.py`, дослідження витоку даних — у `leakage.py`, вкладену крос-валідацію та підсумкове порівняння — у `nested.py`. Сценарій `lab3/run.py` лише запускає етапи по черзі й перевіряє, що кожен записав свої рисунки та метрики. Увесь експеримент відтворюється однією командою з фіксованим зерном генератора випадкових чисел.

1. ВИХІДНІ ДАНІ ТА БАЗОВА МОДЕЛЬ

Використано набір даних і розбиття з лабораторних робіт №1–2: **Diabetes 130-US Hospitals for Years 1999–2008** [1], 69 987 незалежних пацієнтів після очищення. Цільова змінна — повторна госпіталізація протягом 30 днів після виписки (бінарна класифікація), клас 1 становить 8,98 % спостережень. Навчальна вибірка містить 55 989 пацієнтів, тестова — 13 998. Усі дослідження виконано крос-валідацією на навчальній вибірці. Тестову вибірку використано лише для контрольної перевірки висновків у розділах 4 і 6, і жодного вибору на ній не зроблено.

1.1. Вибір моделі

З моделей лабораторної роботи №2 обрано **гістограмний градієнтний бустинг** [4] на даних сценарію Б (повне очищення, Target Encoding високопотужних категоріальних ознак, доменні та агреговані ознаки). Причини вибору:

- у лабораторній роботі №2 він мав найвищу якість ранжування (PR-AUC

0,179–0,180 на крос-валідації);

- він має кілька гіперпараметрів, що взаємодіють: крок навчання, розмір дерева, мінімальний розмір листка та регуляризація. Оптимальне значення одного залежить від інших, тому їх доводиться шукати разом;
- він схильний до перенавчання: за великих дерев і великого кроку навчання ансамбль запам’ятовує навчальні записи;
- він навчається швидко. Значення ознак групуються в 255 інтервалів, тому одне навчання на 37 тисячах записів триває кілька секунд, а вся робота потребує понад три тисячі навчань.

Випадковий ліс мав близьку якість, але його 300 дерев навчаються повільніше, а єдиний гіперпараметр, що суттєво впливав на якість (мінімальний розмір листка), уже досліджено в лабораторній роботі №2.

1.2. Метрика якості

Метрика якості — **PR-AUC** (площа під кривою точність-повнота, у scikit-learn — `average_precision`), основна метрика лабораторних робіт №1–2. Модель видає ймовірність повторної госпіталізації, яку використовують для ранжування пацієнтів за ризиком, тож метрика має оцінювати саме ранжування, без вибору порога. За частки класу 1 близько 9 % PR-AUC чутливіша до якості ранжування позитивного класу, ніж ROC-AUC [5], яку обчислено як допоміжну метрику. Випадковий класифікатор має PR-AUC, що дорівнює частці класу 1 (0,090), і ROC-AUC 0,5. Accuracy непридатна: правило «завжди клас 0» дає 0,910.

1.3. Базова модель

Базова модель — бустинг із гіперпараметрами лабораторної роботи №2: крок навчання 0,05, до 31 листка в дереві, щонайменше 20 записів у листку, L2-регуляризація 1, 250 дерев. Її оцінено двома схемами крос-валідації, з якими далі порівнюються результати оптимізації (табл. 1.1).

Таблиця 1.1. Якість базової моделі

Схема оцінювання	**PR-AUC**	**ROC-AUC**	**Час, с**
3-блокова крос-валідація (як у пошуку)	0,1713 ± 0,0023	0,6491	13,7
5-блокова крос-валідація (як зовнішня петля)	0,1790 ± 0,0052	0,6550	21,3

5-блокова оцінка збігається з результатом лабораторної роботи №2. 3-блокова нижча на 0,008, хоча модель та сама: у 3-блоковій схемі модель навчається на двох третинах навчальної вибірки (37 326 записів), а в 5-блоковій — на чотирьох п’ятих (близько 44 790 записів). Отже, бустинг ще не досяг насичення за обсягом даних, і абсолютні значення метрики в різних схемах не можна порівнювати напряму. Тому далі ефект оптимізації вимірюється як різниця з базовою моделлю в тій самій схемі.

2. ПРОСТІР ГІПЕРПАРАМЕТРІВ І МЕТОДИ ПОШУКУ

2.1. Простір пошуку

Оптимізовано чотири гіперпараметри бустингу, що визначають складність моделі та силу регуляризації (табл. 2.1).

Таблиця 2.1. Простір гіперпараметрів

Гіперпараметр	**Зміст**	**Значення сітки**	**Розподіл для Random і Bayesian**	**Лаб. 2**
`learning_rate`	крок навчання: частка, з якою прогноз нового дерева додається до ансамблю	0,02; 0,063; 0,2	логарифмічно рівномірний на $[0{,}02;\ 0{,}2]$	0,05
`max_leaf_nodes`	найбільша кількість листків у дереві, тобто складність одного дерева	8; 32; 128	цілий, логарифмічно рівномірний на $[8;\ 128]$	31
`min_samples_leaf`	найменша кількість записів у листку	20; 100; 500	цілий, логарифмічно рівномірний на $[20;\ 500]$	20
`l2_regularization`	L2-штраф на значення листків	0,1; 1; 10	логарифмічно рівномірний на $[0{,}1;\ 10]$	1

Для кожного гіперпараметра задано інтервал. Сітка містить його межі та середнє геометричне, тобто $3^4 = 81$ комбінацію. Випадковий і байєсівський пошук обирають точки з тих самих інтервалів, тож усі три методи досліджують одну область і відрізняються лише способом вибору точок у ній. Шкала всюди логарифмічна: вплив цих гіперпараметрів мультиплікативний, і зміна кроку навчання з 0,02 до 0,04 важить стільки ж, скільки з 0,1 до 0,2. Для цілих гіперпараметрів у scikit-learn немає логарифмічного розподілу (`randint` рівномірний на лінійній шкалі й обирав би переважно найбільші та найповільніші дерева), тому його реалізовано окремим класом `LogUniformInt`. Optuna для тих самих параметрів використовує `suggest_int(log=True)`.

Кількість дерев зафіксовано на значенні 250, як у лабораторній роботі

№2, а ранню зупинку вимкнено. Крок навчання і кількість дерев взаємозамінні: менший крок потребує більше дерев. Якщо оптимізувати обидва параметри, час навчання різних кандидатів відрізнявся б у десятки разів, і порівняння методів за часом втратило б сенс. Рання зупинка відділяє частину навчальних записів для внутрішньої перевірки, тож кандидати навчалися б на різних даних.

2.2. Методи пошуку

- **Повний перебір** (`GridSearchCV`) оцінює всі 81 комбінацію сітки.
- **Випадковий пошук** (`RandomizedSearchCV`) [6] оцінює $N = 20$ та $N = 50$ точок, вибраних незалежно з розподілів табл. 2.1. Обидва запуски використовують те саме зерно, а точки обираються послідовно одним генератором, тож 20 точок першого запуску збігаються з першими 20 точками другого. Різниця між ними показує внесок саме додаткових 30 точок.
- **Байєсівська оптимізація** [8] реалізована бібліотекою Optuna з алгоритмом TPE (Tree-structured Parzen Estimator) [7], $N = 20$ спроб. TPE ділить уже оцінені точки на кращі та гірші й обирає наступну там, де відношення щільностей цих двох груп найбільше. Перші 5 спроб випадкові: моделі потрібні результати, на яких вона навчиться. Типове значення Optuna (10) витратило б на випадкові точки половину бюджету в 20 спроб. Використано багатовимірний TPE, що моделює гіперпараметри спільно: крок навчання і розмір дерева мають сенс лише разом.

Повний перебір і випадковий пошук виконують класи scikit-learn з однаковими параметрами оцінювання (лістинг 1), байєсівську оптимізацію — цільова функція Optuna, що оцінює кандидата тією самою крос-валідацією (лістинг 2). Кожен метод повертає історію всіх оцінених кандидатів, з якої обирається кандидат із найвищою середньою PR-AUC. Пошук не перенавчає остаточну модель (`refit=False`): час пошуку — це час вибору гіперпараметрів.

```
    common = {
        "scoring": SCORING,
        "refit": False,
        "cv": inner_folds(),
        "n_jobs": N_JOBS,
    }
    if method.kind == "grid":
        grid = {f"model__{name}": values for name, values in GRID.items()}
        searcher = GridSearchCV(build_tuned({}), grid, **common)
    else:
        searcher = RandomizedSearchCV(
            build_tuned({}),
            distributions(),
            n_iter=method.n_iter,
            random_state=SEED,
            **common,
        )
    searcher.fit(X, y)
```

Лістинг 1. Повний перебір і випадковий пошук (`src/mk1/tuning.py`)

```
    folds = inner_folds()
    history: list[dict] = []

    def objective(trial: Trial) -> float:
        params = {name: suggest(trial, name) for name in GRID}
        scores = cross_validate(
            build_tuned(params), X, y, scoring=SCORING, cv=folds, n_jobs=N_JOBS
        )
        history.append(candidate(params, scores))
        log.debug("Trial %d: %s", trial.number, history[-1])
        return history[-1][SELECT]

    # multivariate=True is Optuna 5's default for one objective; it is spelled
    # out because the learning rate and the tree size only make sense together.
    sampler = TPESampler(n_startup_trials=STARTUP_TRIALS, multivariate=True, seed=SEED)
    study = create_study(direction="maximize", sampler=sampler)
    study.optimize(objective, n_trials=method.n_iter)
```

Лістинг 2. Байєсівська оптимізація з Optuna (`src/mk1/tuning.py`)

2.3. Оцінювання кандидатів і вимірювання часу

Кожного кандидата оцінено стратифікованою 3-блоковою крос-валідацією на всій навчальній вибірці. Блоки однакові для всіх методів, тож відмінності між методами зумовлені лише вибором кандидатів. На кожному блоці заново навчається весь конвеєр, разом із Target Encoding і масштабуванням, а не

лише класифікатор. Ця ж 3-блокова схема є внутрішньою петлею вкладеної крос-валідації в розділі 5, тому найкраща оцінка пошуку та внутрішня оцінка вкладеної схеми виміряні однаково.

Час виміряно двома способами:

- **тривалість пошуку** — час від початку до кінця пошуку на комп’ютері з чотирма ядрами;
- **сумарний обчислювальний час** — сума тривалостей усіх навчань і оцінювань кандидатів на всіх блоках. Він не залежить від того, скільки навчань виконувалося паралельно, і пропорційний обсягу роботи методу.

Ці величини відрізняються через паралелізм. `GridSearchCV` і `RandomizedSearchCV` знають усіх кандидатів наперед і навчають їх у чотирьох процесах одночасно. Байєсівська оптимізація обирає наступну точку за результатами всіх попередніх, тому спроби виконуються одна за одною, і паралельно навчаються лише три блоки однієї спроби. Отже, байєсівський пошук використовує три ядра з чотирьох. Це властивість методу, а не реалізації, і її враховано у висновках.

3. ДОСЛІДЖЕННЯ 1: ПОРІВНЯННЯ МЕТОДІВ ОПТИМІЗАЦІЇ

Кожен метод виконано на всій навчальній вибірці. Результати наведено в табл. 3.1, обрані гіперпараметри — в табл. 3.2. Приріст обчислено відносно базової моделі в тій самій 3-блоковій схемі (PR-AUC 0,1713).

Таблиця 3.1. Порівняння методів пошуку на 3-блоковій крос-валідації

Метод	**Кандидатів**	**Тривалість, с**	**Обчисл. час, с**	**Найкраща PR-AUC**	**Приріст**	**ROC-AUC**
Без оптимізації (лаб. 2)	1	14	30	0,1713 ± 0,0023	—	0,6491
Grid Search	81	766	2972	0,1783 ± 0,0048	+0,0070	0,6562
Random Search, $N = 20$	20	182	685	0,1770 ± 0,0059	+0,0057	0,6564
Random Search, $N = 50$	50	459	1761	0,1770 ± 0,0059	+0,0057	0,6564
Bayesian (TPE), $N = 20$	20	220	612	0,1779 ± 0,0045	+0,0066	0,6565

Таблиця 3.2. Гіперпараметри, обрані кожним методом

Метод	**Крок навчання**	**Листків у дереві, до**	**Записів у листку, від**	**L2-штраф**
Без оптимізації (лаб. 2)	0,05	31	20	1
Grid Search	0,02	32	100	10
Random Search, $N = 20$ і $N = 50$	0,033	13	36	4,62
Bayesian (TPE), $N = 20$	0,020	18	110	2,39

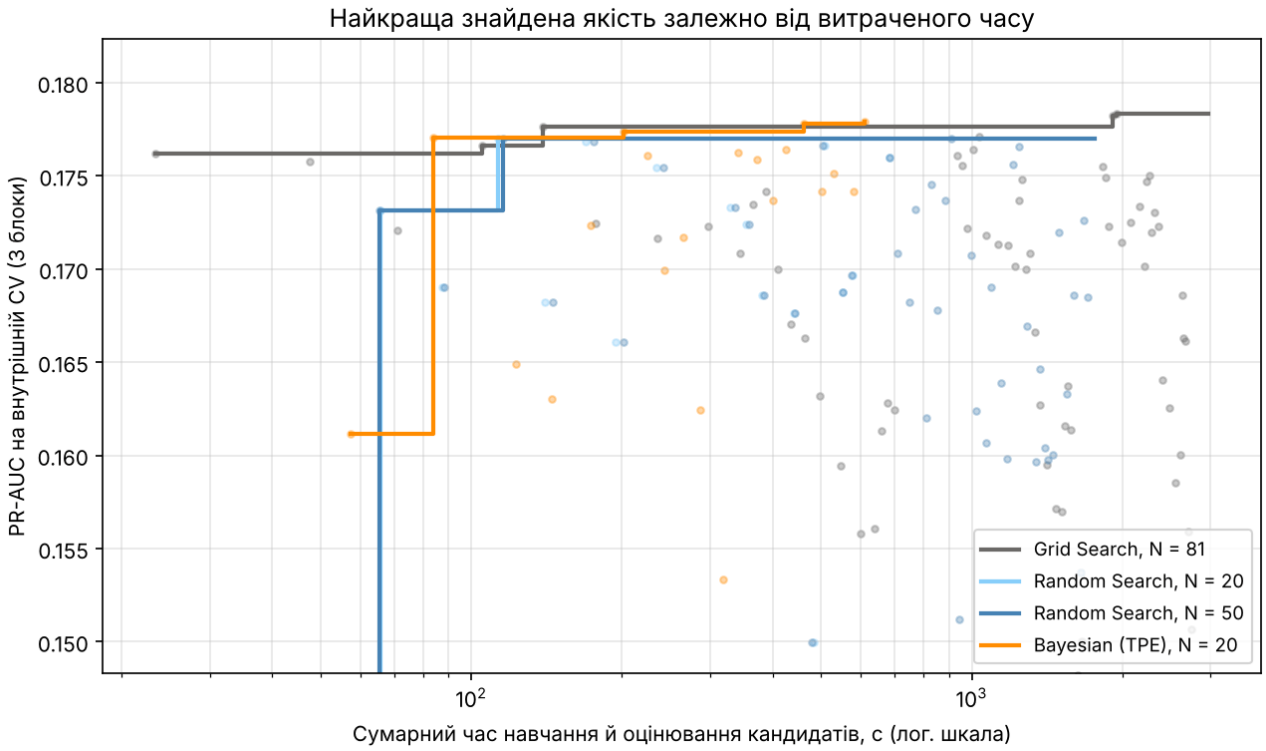


Рис. 3.1. Найкраща PR-AUC, знайдена до певного моменту, залежно від сумарного обчислювального часу. Точки — окремі кандидати в порядку їх оцінювання; кандидати з PR-AUC нижче 0,148 за межами графіка

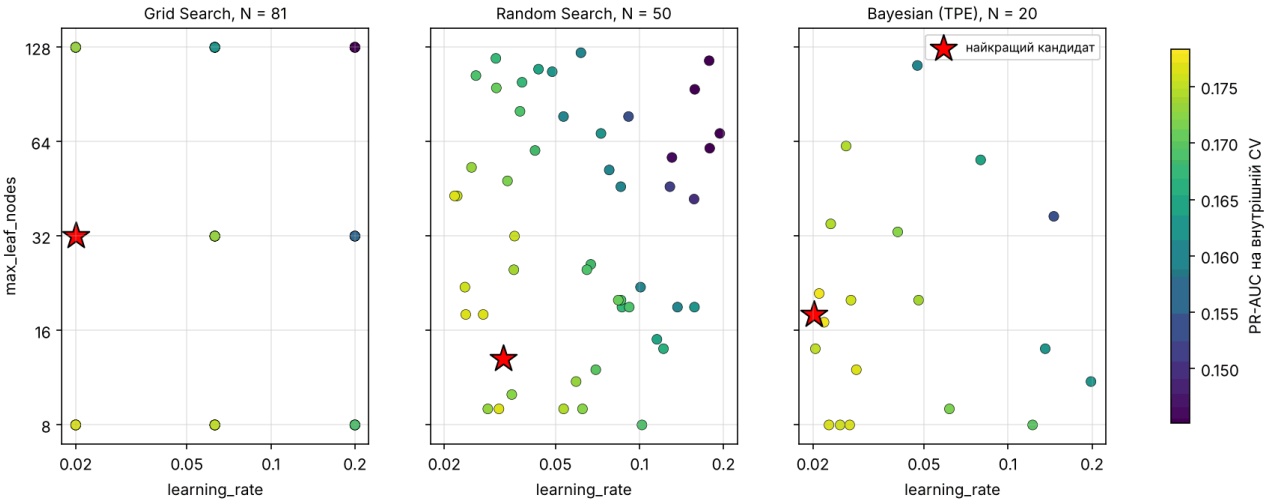


Рис. 3.2. Точки, які оцінив кожен метод, у площині «крок навчання — кількість листків», колір — PR-AUC. У повному переборі кожна точка площини поєднує 9 кандидатів з різними значеннями двох інших гіперпараметрів, зверху показано найкращого з них

Усі методи покращили модель приблизно однаково. Приріст становить 0,006–0,007 PR-AUC, тобто 3–4 % від базового значення. Найкращі оцінки методів відрізняються не більше ніж на 0,0013, а стандартне відхилення

найкращого кандидата між блоками становить 0,005. Отже, за якістю методи в межах похибки еквівалентні, і різниця між ними — це насамперед вартість. Поверхня якості пласка поблизу оптимуму: у кожного методу від 6 до 8 кандидатів відстають від найкращого менше ніж на 0,002 PR-AUC.

Усі методи обрали менший крок навчання, ніж у лабораторній роботі №2 (0,02–0,033 замість 0,05), і сильнішу регуляризацію: більший мінімальний розмір листка (36–110 записів замість 20) та більший L2-штраф (2,4–10 замість 1). Отже, базова модель зі 250 деревами та кроком 0,05 дещо перенавчена. Grid Search і TPE обрали крок 0,02, тобто нижню межу інтервалу пошуку. Оптимум може лежати нижче, але менший крок потребує більше дерев і, відповідно, більше часу на кожного кандидата. Це обмеження заданого простору пошуку, а не методів.

Повний перебір знайшов найвищу оцінку (0,1783), але тривав у 3,5 раза довше за байєсівську оптимізацію й виконав у 4,9 раза більше обчислень заради різниці в 0,0004, що значно менша за похибку. Сітка має 81 точку, але лише по три різні значення кожного гіперпараметра (рис. 3.2, ліворуч): якщо важливий лише один параметр, 81 оцінка дає про нього стільки ж інформації, скільки три [6]. На рис. 3.1 крива повного перебору рано досягає високого рівня лише тому, що перші кандидати сітки мають найменший крок навчання, який і виявився найкращим. За іншого порядку значень перебір довго оцінював би гірші точки. Вартість повного перебору зростає експоненційно: п’ятий гіперпараметр із трьома значеннями збільшив би її втричі, до 243 кандидатів.

Випадковий пошук знайшов свого найкращого кандидата вже четвертою точкою, приблизно за 2 хвилини обчислювального часу. Подальші 16 точок ($N = 20$), а потім ще 30 ($N = 50$) його не покращили: лише одна з нових точок дала майже такий самий результат. Збільшення бюджету в 2,5 раза не дало нічого, бо пласку область поблизу оптимуму випадкові точки вже знайшли, а щоб потрапити ще ближче до межі інтервалу, де лежить оптимум, потрібне везіння, а не кількість спроб. Випадкові точки рівномірно вкривають увесь простір (рис. 3.2, у центрі), і 50 різних значень кожного гіперпараметра дали б

перевагу над сіткою, якби якість залежала від одного з них різко.

Байєсівська оптимізація покращувала результат протягом усього бюджету: 0,1770 після другої спроби, 0,1774 після шостої, 0,1778 після шістнадцятої і 0,1779 на останній, двадцятій. TPE швидко зосередився на області малого кроку навчання й невеликих дерев (рис. 3.2, праворуч): 14 з 20 його кандидатів кращі за базову модель, проти 16 з 50 у випадкового пошуку та 32 з 81 у сітці. Медіана PR-AUC його кандидатів — 0,1742, у випадкового пошуку — 0,1684, у сітки — 0,1671. TPE також витратив найменше обчислювального часу (612 с), бо обирав невеликі дерева, які навчаються швидше. Проте за тривалістю він повільніший за випадковий пошук з тим самим бюджетом (220 с проти 182 с): спроби виконуються послідовно, і з чотирьох ядер використовуються лише три. Відношення обчислювального часу до тривалості становить 2,8 для TPE проти 3,8–3,9 для інших методів.

4. ДОСЛІДЖЕННЯ 2: ВПЛИВ ВИТОКУ ДАНИХ

4.1. Варіанти конвеєра

Витік даних (data leakage) — це ситуація, коли модель під час навчання чи вибору отримує інформацію, якої не матиме під час застосування до нових записів [9]. Під час крос-валідації найчастіше джерело витоку — кроки підготовки даних, навчені на всій вибірці ще до її поділу на блоки. Тоді валідаційний блок уже «бачили» кроки, що його перетворюють, і оцінка якості більше не відповідає ситуації з новим пацієнтом.

Порівняно чотири варіанти (лістинг 3). У всіх варіантах ті самі записи, та сама модель (бустинг із гіперпараметрами лабораторної роботи №2) і ті самі 5 блоків крос-валідації. Відрізняється лише те, які кроки навчаються на всій навчальній вибірці до поділу на блоки.

1. **Коректний конвеєр.** Увесь конвеєр, від інженерії ознак до класифікатора, передано в крос-валідацію. На кожному навчальному блоці всі кроки навчаються заново, а до валідаційного блоку лише

застосовуються.

2. **Препроцесинг до розбиття (`fit_transform`).** Усі кроки підготовки навчено на всій навчальній вибірці методом `fit_transform`, а в крос-валідацію передано лише класифікатор. Групові середні агрегатних ознак, моди для пропусків і параметри масштабування обчислено разом із валідаційними записами. Target Encoding у `fit_transform` бібліотеки scikit-learn навчається з перехресним навчанням: кожен рядок кодується середніми, обчисленими на інших блоках, тобто без власної мітки.
3. **Препроцесинг до розбиття (`fit`, потім `transform`).** Ті самі кроки навчено методом `fit` на всій навчальній вибірці, яку потім перетворено методом `transform`. Це звичний рецепт для масштабування, але для Target Encoding він означає, що кожен рядок закодовано середнім значенням цілі, обчисленим разом із його власною міткою. Саме цей варіант є некоректним конвеєром, якого вимагає завдання.
4. **Препроцесинг і SMOTE до розбиття.** Варіант 2, після якого всю навчальну вибірку доповнено синтетичними записами класу 1 методом SMOTE [11] до співвідношення 1 : 1, і лише потім поділено на блоки.

```
64  def prefit_fit_transform(
65      X: pd.DataFrame, y: pd.Series, X_test: pd.DataFrame
66  ) -> Matrices:
67      """Preprocessing learned on every training row via fit_transform (cross-fitted
        ↪  encoding)."""
68      pipe = build_tuned({})
69      head, model = pipe[:-1], pipe.named_steps["model"]
70      Xt = head.fit_transform(X, y)
71      return Matrices(model, Xt, y, head.transform(X_test))
72
73
74  def prefit_fit_then_transform(
75      X: pd.DataFrame, y: pd.Series, X_test: pd.DataFrame
76  ) -> Matrices:
77      """Preprocessing fitted on every training row, which are then transformed with it."""
78      pipe = build_tuned({})
79      head, model = pipe[:-1], pipe.named_steps["model"]
80      # fit, then transform: each row is target-encoded with means that include
81      # its own label — the leak this variant exists to measure.
82      head.fit(X, y)
83      return Matrices(model, head.transform(X), y, head.transform(X_test))
```

```
84
85
86  def prefit_with_smote(X: pd.DataFrame, y: pd.Series, X_test: pd.DataFrame) -> Matrices:
87      """The fit_transform variant, then SMOTE over-sampling of every training row."""
88      pipe = build_tuned({})
89      head, model = pipe[:-1], pipe.named_steps["model"]
90      Xt = head.fit_transform(X, y)
91      Xs, ys = make_sampler("smote").fit_resample(Xt, y)
92      return Matrices(model, Xs, ys, head.transform(X_test))
```

Лістинг 3. Підготовка даних у варіантах із витоком (`src/mk1/leakage.py`)

Кожен варіант також навчено на всій навчальній вибірці й оцінено на тестовій. Тестова вибірка не брала участі в навчанні жодного кроку жодного варіанта, тож її оцінка чесна. Різниця між оцінкою крос-валідації та тестовою показує, наскільки витік завищив оцінку.

4.2. Результати

Результати наведено в табл. 4.1 та на рис. 4.1. Стовпець «Завищення» — різниця між оцінкою крос-валідації варіанта та коректного конвеєра. Стовпець «Зміна на тесті» — така сама різниця на тестовій вибірці, тобто реальна зміна якості моделі.

Таблиця 4.1. Оцінки якості за різної підготовки даних відносно поділу на блоки

	PR-AUC				ROC-AUC	
Варіант	**CV**	**Завищення**	**Тест**	**Зміна на тесті**	**CV**	**Тест**
1. Коректний конвеєр	0,179 ± 0,005	—	0,189	—	0,655	0,654
2. Препроцесинг до розбиття (`fit_transform`)	0,182 ± 0,005	+0,003	0,189	0,000	0,655	0,654
3. Препроцесинг до розбиття (`fit` + `transform`)	0,230 ± 0,008	+0,051	0,161	−0,028	0,711	0,633
4. Препроцесинг і SMOTE до розбиття	0,885 ± 0,052	+0,706	0,101	−0,088	0,829	0,516

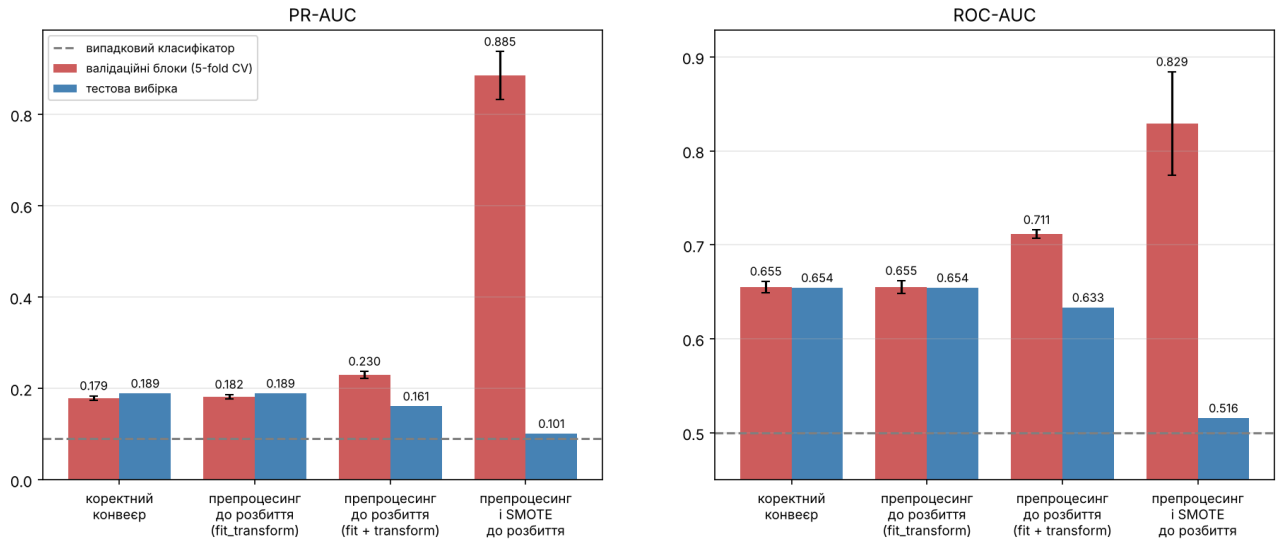


Рис. 4.1. Оцінка крос-валідації (середнє ± стандартне відхилення за 5 блоками) і тестова оцінка кожного варіанта. Пунктир — випадковий класифікатор на реальних даних. У варіанті 4 валідаційні блоки наполовину складаються з синтетичних записів, і для них PR-AUC випадкового класифікатора становить 0,5

Некоректний конвеєр (варіант 3) завищив PR-AUC на 0,051, тобто на 28 %: крос-валідація показує 0,230 замість 0,179, а ROC-AUC — 0,711 замість 0,655. Розкид між блоками (0,008) у шість разів менший за завищення, тож за самими результатами крос-валідації помилку не видно: дослідник просто вирішив би, що модель стала кращою. Насправді модель стала гіршою. На тестовій вибірці її PR-AUC 0,161 проти 0,189 у коректного конвеєра, а ROC-AUC 0,633 проти 0,654.

Механізм витоку пов’язаний із високопотужними ознаками. Коди діагнозів `diag_1`–`diag_3` мають 671–731 різних значень у навчальній вибірці, і більше половини з них трапляються не більше ніж у 10 пацієнтів (91–141 код — лише в одного). Target Encoding замінює категорію середнім значенням цілі за її записами. Коли це середнє обчислено разом із власною міткою рядка, для рідкісного коду воно майже дорівнює цій мітці: пацієнт з унікальним кодом і повторною госпіталізацією отримує кодування, близьке до 1. Згладжування в `TargetEncoder` лише наближає таке кодування до загального середнього, але не прибирає залежність від власної мітки. Бустинг знаходить цю залежність,

бо вона найсильніша в даних, і на валідаційних блоках вона так само працює: їхні записи закодовано з урахуванням їхніх власних міток. Новий пацієнт такої підказки не має, тож модель, що покладається на неї, на тесті прогнозує гірше за модель, навчену на чесному кодуванні.

Варіант 2 майже не завищує оцінку (+0,003, менше за розкид між блоками), а остаточна модель у ньому та сама, що й у коректному конвеєрі, тому тестові оцінки збігаються. Причина в тому, що `TargetEncoder.fit_transform` у scikit-learn кодує кожен рядок середніми, обчисленими на інших блоках, тобто саме для того, щоб власна мітка не потрапила в кодування. Решта кроків, навчених до розбиття (групові середні, моди, параметри масштабування), цілі не використовують, і для деревоподібної моделі їхній витік несуттєвий. Отже, небезпечний не будь-який препроцесинг до розбиття, а той, що навчається на цілі. Проте покладатися на цей захист не варто: він працює лише за виклику `fit_transform`, і варіант 3 відрізняється від варіанта 2 тільки тим, що `fit` і `transform` викликано окремо.

SMOTE до розбиття (варіант 4) дає найбільше завищення: PR-AUC 0,885 на крос-валідації проти 0,101 на тесті, а ROC-AUC 0,829 проти 0,516, що майже не відрізняється від випадкового ранжування. Тут поєднано два ефекти. По-перше, валідаційні блоки наполовину складаються з синтетичних записів класу 1, тож частка класу 1 у них 50 % замість 9 %, і PR-AUC навіть випадкового класифікатора зростає до 0,5. По-друге, кожен синтетичний запис лежить на відрізку між двома реальними записами класу 1, які здебільшого опинилися в навчальних блоках. Модель розпізнає синтетичні записи як «майже копії» навчальних. У лабораторній роботі №2 показано, що деревоподібні моделі розпізнають синтетичні записи майже безпомилково за дробовими значеннями закодованих ознак. Остаточна модель варіанта 4, навчена на всій навчальній вибірці, збігається з моделлю сценарію B (SMOTE) лабораторної роботи №2, і її ROC-AUC на тесті та сама: 0,516. Там передискретизацію виконано коректно, лише на навчальних блоках, і крос-валідація чесно показала падіння якості.

Отже, SMOTE шкодить цій моделі і без витоку, а витік не просто завищує оцінку, а приховує цю шкоду, показуючи майже ідеальну якість.

Висновок дослідження: оцінка крос-валідації коректна лише тоді, коли всі кроки, що навчаються на даних, особливо ті, що використовують ціль (Target Encoding, передискретизація, відбір ознак), знаходяться всередині конвеєра й навчаються заново на кожному блоці. Розкид між блоками не виявляє витоку. Виявити його можна лише порівнянням з оцінкою на даних, які жоден крок не бачив.

5. ДОСЛІДЖЕННЯ 3: ВКЛАДЕНА КРОС-ВАЛІДАЦІЯ

5.1. Схема

Найкраща оцінка пошуку з розділу 3 зміщена вгору. Пошук оцінює десятки кандидатів на тих самих блоках і обирає максимум. Кожна оцінка містить випадкову похибку, тож максимум відбирає не лише кращий набір гіперпараметрів, а й найвдалішу похибку [12, 13]. Чим більше кандидатів, тим сильніший цей ефект. Щоб отримати незміщену оцінку, вибір гіперпараметрів і оцінювання мають використовувати різні дані. Це забезпечує **вкладена крос-валідація** (лістинг 4):

- **зовнішня петля** ділить навчальну вибірку на $K = 5$ стратифікованих блоків. Кожен блок по черзі стає тестовим для оцінки узагальнювальної здатності, а решта чотири — навчальними;
- **внутрішня петля** на навчальних блоках зовнішньої петлі виконує весь пошук гіперпараметрів з 3-блоковою крос-валідацією, так само як у розділі 3;
- обрані гіперпараметри навчаються на всіх чотирьох навчальних блоках зовнішньої петлі й один раз оцінюються на п’ятому, якого пошук не бачив.

Середнє п’яти зовнішніх оцінок — незміщена оцінка якості *методу* пошуку, а не одного обраного кандидата: на кожному зовнішньому блоці метод може обрати інші гіперпараметри. Для кожного методу виконано окрему вкладену

крос-валідацію з тими самими блоками.

```
    folds = []
    for k, (train_idx, valid_idx) in enumerate(outer_folds().split(X, y)):
        X_in, y_in = X.iloc[train_idx], y.iloc[train_idx]
        result = search(method, X_in, y_in)
        outer = holdout_scores(
            result.best_params, X_in, y_in, X.iloc[valid_idx], y.iloc[valid_idx]
        )
        folds.append(
            {
                "inner": result.best_score,
                "outer": outer[SELECT],
                "outer_roc_auc": outer["roc_auc"],
                "params": result.best_params,
                "wall_seconds": result.wall_seconds,
                "compute_seconds": result.compute_seconds,
            }
        )
```

Лістинг 4. Вкладена крос-валідація одного методу пошуку (`src/mk1/nested.py`)

Внутрішні оцінки отримано на моделях, навчених на двох третинах зовнішньої навчальної частини (близько 29,9 тис. записів), а зовнішні — на всій частині (близько 44,8 тис.). Табл. 1.1 показує, що вже ця різниця в обсязі даних змінює PR-AUC. Тому базову модель без оптимізації оцінено на тих самих внутрішніх і зовнішніх блоках. Її різниця між внутрішньою та зовнішньою оцінкою — це ефект обсягу даних, а різниця методу понад неї — оптимізм вибору.

5.2. Результати

Результати наведено в табл. 5.1 та на рис. 5.1. «Приріст» — різниця з базовою моделлю на тих самих блоках, для зовнішньої оцінки обчислена окремо для кожного зовнішнього блоку (середнє $\pm$ стандартне відхилення за 5 блоками). «Оптимізм вибору» — різниця між приростом, який обіцяє внутрішня крос-валідація, і приростом, отриманим на зовнішніх блоках.

Таблиця 5.1. Вкладена крос-валідація: внутрішні та зовнішні оцінки PR-AUC

Метод	Внутрішня CV	Приріст	Зовнішня CV (незміщена)	Приріст	Оптимізм вибору	Час, с
Без оптимізації	0,1680	—	0,1790 ± 0,0052	—	—	—
Grid Search	0,1753 ± 0,0021	+0,0072	0,1818 ± 0,0043	+0,0029 ± 0,0033	0,0044	2862
Random Search, $N = 20$	0,1744 ± 0,0027	+0,0064	0,1828 ± 0,0046	+0,0038 ± 0,0015	0,0026	655
Random Search, $N = 50$	0,1745 ± 0,0027	+0,0064	0,1833 ± 0,0041	+0,0043 ± 0,0020	0,0021	1672
Bayesian (TPE), $N = 20$	0,1751 ± 0,0016	+0,0071	0,1822 ± 0,0047	+0,0032 ± 0,0043	0,0039	785

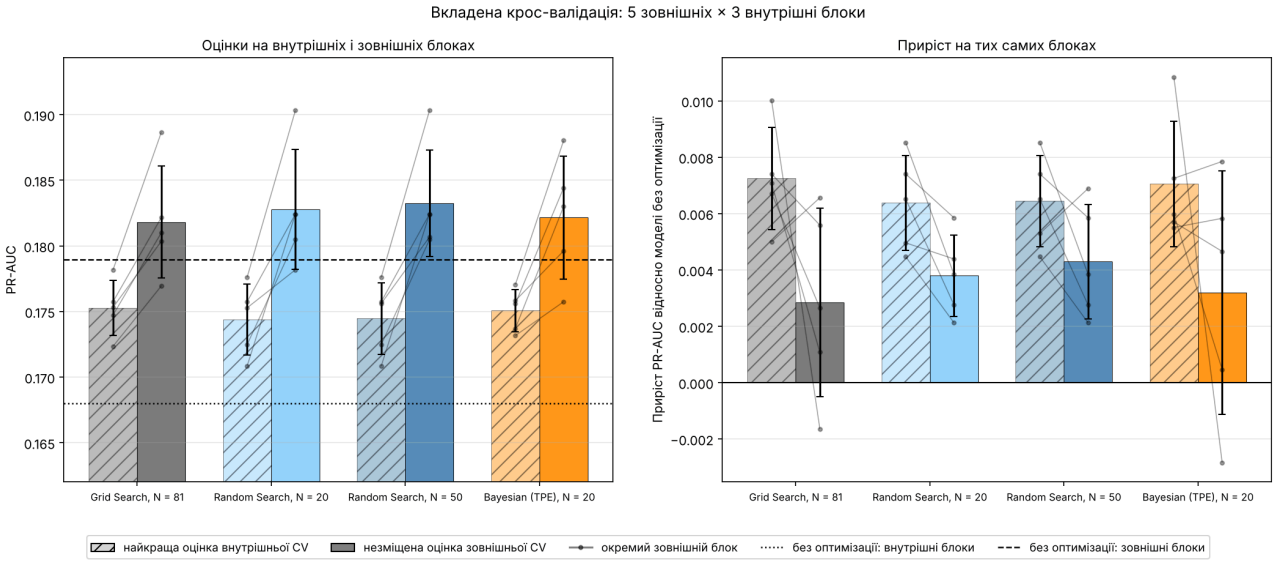


Рис. 5.1. Вкладена крос-валідація кожного методу. Ліворуч — найкраща внутрішня та незміщена зовнішня оцінка PR-AUC, праворуч — їхній приріст відносно моделі без оптимізації на тих самих блоках. Лінії з'єднують оцінки одного зовнішнього блоку, стовпці — середнє ± стандартне відхилення за 5 блоками

Без поправки на обсяг даних результат виглядає парадоксально: зовнішня оцінка кожного методу *вища* за внутрішню на 0,007–0,009, тобто незміщена оцінка ніби перевищує зміщену вгору. Модель без оптимізації пояснює причину: на тих самих блоках її зовнішня оцінка вища за внутрішню на 0,011 лише тому, що модель навчається на 44,8 тис. записів замість 29,9 тис. Якщо порівнювати приріст відносно цієї моделі, картина протилежна. Внутрішня крос-валідація обіцяє від оптимізації +0,006–0,007 PR-AUC, а зовнішні блоки дають лише +0,003–0,004. Отже, від третини до 61 % приросту, який бачить пошук, становить оптимізм вибору, а не покращення моделі.

Оптимізм найбільший у повного перебору та байєсівської оптимізації (0,0044 і 0,0039), а у випадкового пошуку він найменший (0,0021–0,0026). Повний перебір обирає максимум з 81 оцінки, і серед такої кількості майже рівноцінних кандидатів завжди знайдеться той, кому пощастило з блоками. TPE оцінює лише 20 кандидатів, але цілеспрямовано скупчує їх біля найкращої області (розділ 3). Отже, він обирає максимум серед багатьох майже однакових оцінок, і вибір значною мірою визначає випадкова похибка. Випадкові точки розкидані по всьому простору, тому біля вершини за перше місце змагаються лише кілька з них. Ефективність TPE у пошуку високої внутрішньої оцінки частково полягає саме в тому, що він знаходить кандидатів, яким пощастило з конкретними блоками.

Оптимізація все ж дає реальне покращення. Приріст на зовнішніх блоках додатний у середньому для всіх методів, а для випадкового пошуку — на кожному з п’яти блоків (від +0,002 до +0,007). У повного перебору й TPE на першому зовнішньому блоці приріст від’ємний ($-0{,}0016$ і $-0{,}0028$). Обидва методи обрали там найменші дерева (8 листків), що перемогли на внутрішніх блоках, але програли базовій моделі на зовнішньому.

Обрані гіперпараметри нестабільні. Повний перебір на всіх п’яти зовнішніх блоках обрав крок навчання 0,02, але різні значення інших параметрів: 8 або 32 листки, мінімальний розмір листка 20 або 100, L2-регуляризацію від 0,1 до 10. Випадковий пошук тричі з п’яти обрав одного й того самого кандидата, а $N = 50$

змінив вибір порівняно з $N = 20$ лише на одному блоці. TPE на кожному блоці обрав іншу комбінацію, хоча всі вони лежать у вузькій області: крок 0,022–0,036, 8–36 листків. Якщо різні комбінації майже однаково добрі, точне налаштування мало що дає, а вибір між ними визначає шум.

Порядок методів змінився. За найкращою внутрішньою оцінкою (табл. 3.1) перший повний перебір, далі TPE і випадковий пошук. За незміщеною оцінкою все навпаки: перший випадковий пошук ($N = 50$, потім $N = 20$), далі TPE, останній повний перебір. Різниці між методами (до 0,0015) менші за розкид парних приростів між блоками (0,0015–0,0043), тож жоден метод не кращий статистично значуще. Але якби метод обирали за найкращою внутрішньою оцінкою, перевагу отримав би той, чия оцінка найоптимістичніша.

Вкладена крос-валідація коштує приблизно стільки, скільки п’ять пошуків: від 11 хвилин для випадкового пошуку з $N = 20$ до 48 хвилин для повного перебору. Кожен пошук виконується на 80 % даних, тому загальний час у 3,6–3,7 раза більший за один пошук на всій навчальній вибірці, а не в 5 разів.

6. ПІДСУМКОВЕ ПОРІВНЯННЯ

Підсумкову таблицю у форматі, якого вимагає завдання, наведено в табл. 6.1, а залежність якості від часу — на рис. 6.1. Кількість оцінок дорівнює кількості навчань моделі під час пошуку: кожен кандидат навчається на 3 блоках. Час — тривалість пошуку на всій навчальній вибірці. Додатково наведено PR-AUC моделі, навченої з обраними гіперпараметрами на всій навчальній вибірці й один раз оціненої на тестовій.

Таблиця 6.1. Порівняльна таблиця методів оптимізації

Метод оптимізації	**Кількість ітерацій / оцінок**	**Час виконання, с**	**Best Score (Inner CV)**	**Unbiased Score (Nested CV)**	**Тест**
Без оптимізації (лаб. 2)	1 / 3	14	0,1713	0,1790 ± 0,0052	0,1892
Grid Search	81 / 243	766	0,1783	0,1818 ± 0,0043	0,1931
Random Search, $N = 20$	20 / 60	182	0,1770	0,1828 ± 0,0046	0,1957
Random Search, $N = 50$	50 / 150	459	0,1770	0,1833 ± 0,0041	0,1957
Bayesian (TPE), $N = 20$	20 / 60	220	0,1779	0,1822 ± 0,0047	0,1934

Три стовпці якості виміряно на моделях, навчених на різних обсягах даних: 37,3 тис. записів у внутрішній CV, близько 44,8 тис. у Nested CV і 56,0 тис. для тесту. Тому вони зростають зліва направо в кожному рядку, і порівнювати їх слід усередині стовпця, а не вздовж рядка. Приріст відносно моделі без оптимізації в тій самій схемі наведено в табл. 3.1 і 5.1.

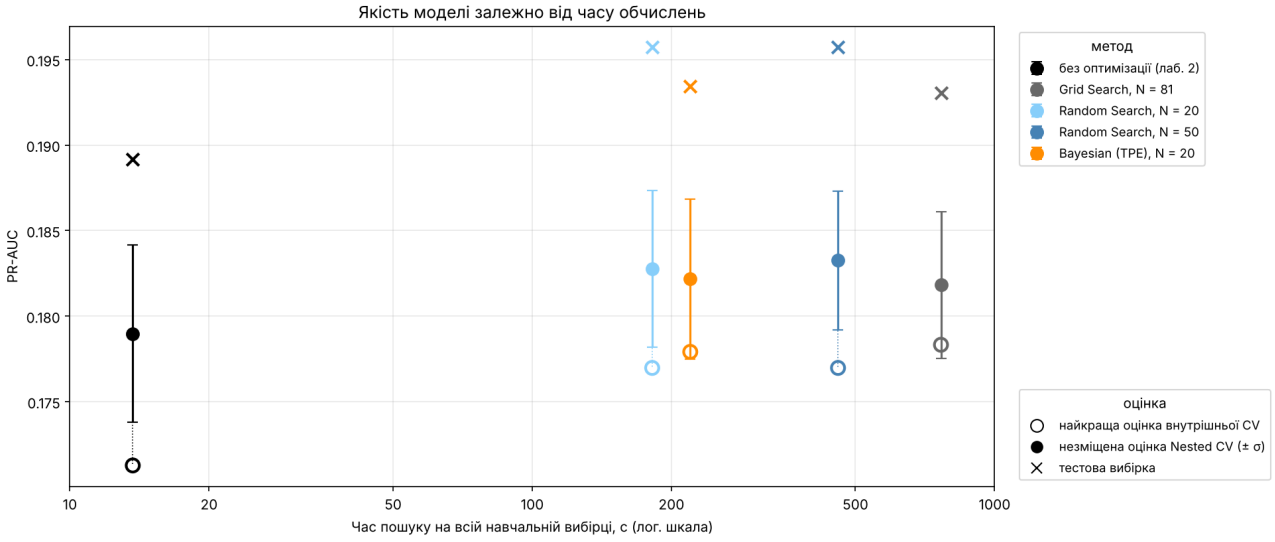


Рис. 6.1. Якість моделі залежно від часу пошуку: найкраща оцінка внутрішньої CV, незміщена оцінка Nested CV (середнє ± стандартне відхилення за 5 зовнішніми блоками) і оцінка на тестовій вибірці

Тестова вибірка підтвердила висновки вкладеної крос-валідації. Модель кожного методу краща на тесті за модель без оптимізації, на 0,004–0,007 PR-AUC, що близько до приросту на зовнішніх блоках (0,003–0,004). Порядок методів на тесті такий самий, як за Nested CV: випадковий пошук (обидва бюджети обрали одну модель) попереду, повний перебір позаду, і він протилежний порядку за найкращою внутрішньою оцінкою. Утім, тест — це одна вибірка з 1 257 випадками класу 1, і різниці між методами на ньому (до 0,0026) не більші за різниці між зовнішніми блоками.

Рис. 6.1 показує, що час пошуку визначає не якість моделі, а лише вартість її отримання. Від 182 с до 766 с незміщена оцінка не зростає, а розкид між блоками в кожного методу приблизно втричі більший за різницю між найкращим і найгіршим із них. Найкраще співвідношення якості та часу має випадковий пошук з $N = 20$: та сама незміщена якість, що й у повного перебору, за 24 % його часу та 23 % обчислень.

7. ВИСНОВКИ

У лабораторній роботі для гістограмного градієнтного бустингу з лабораторної роботи №2 (задача передбачення повторної госпіталізації, Diabetes 130-US Hospitals) порівняно методи оптимізації гіперпараметрів, оцінено вплив витоку даних і отримано незміщену оцінку якості вкладеною крос-валідацією.

1. Оптимізація чотирьох гіперпараметрів покращила PR-AUC бустингу, але небагато. На внутрішній 3-блоковій крос-валідації приріст становив +0,006–0,007 (з 0,1713 до 0,1770–0,1783), незміщена оцінка вкладеної крос-валідації показала +0,003–0,004 (з 0,1790 до 0,1818–0,1833), тестова вибірка — +0,004–0,007. Усі методи обрали менший крок навчання (0,02–0,033 замість 0,05) і сильнішу регуляризацію: модель лабораторної роботи №2 була дещо перенавчена.
2. За якістю всі три методи пошуку в межах похибки еквівалентні: найкращі оцінки відрізняються на 0,0013 на внутрішній CV і на

0,0015 на Nested CV, а стандартне відхилення між блоками становить 0,002–0,005. Відрізняється вартість. Повний перебір 81 комбінації тривав 766 с і виконав 2972 с обчислень, випадковий пошук з $N = 20$ — 182 с і 685 с, TPE з $N = 20$ — 220 с і 612 с. Збільшення бюджету випадкового пошуку з 20 до 50 точок не змінило найкращого кандидата. TPE зосередився на кращій області простору (70 % його кандидатів кращі за базову модель проти 32–40 % в інших методів), але через послідовність спроб використав лише три ядра з чотирьох.

3. Некоректний конвеєр, у якому препроцесинг навчено на всій навчальній вибірці до поділу на блоки (`fit`, потім `transform`), завищив PR-AUC крос-валідації на 0,051 (з 0,179 до 0,230, на 28 %), хоча на тестовій вибірці його модель гірша (0,161 проти 0,189). Причина — Target Encoding рідкісних кодів діагнозів середнім, обчисленим разом із власною міткою рядка. SMOTE до розбиття завищив PR-AUC до 0,885 за тестової 0,101, тобто приховав, що з SMOTE модель ранжує майже випадково. Той самий препроцесинг через `fit_transform`, що кодує рядки з перехресним навчанням, майже не завищує оцінку (+0,003). Розкид між блоками не виявляє жодного з цих витоків.
4. Найкраща оцінка пошуку зміщена вгору. Після поправки на обсяг навчальних даних від 33 до 61 % приросту, який бачить внутрішня крос-валідація, становить оптимізм вибору. Оптимізм найбільший у повного перебору (0,0044, найбільше кандидатів) і TPE (0,0039, кандидати скупчені біля вершини), найменший — у випадкового пошуку (0,0021–0,0026). Порівняння внутрішніх і зовнішніх оцінок без поправки вводить в оману: зовнішні оцінки вищі лише тому, що модель навчається на більшій частині даних.
5. Ранжування методів за найкращою внутрішньою оцінкою (Grid, TPE, Random) протилежне ранжуванню за незміщеною оцінкою та за тестом (Random, TPE, Grid). Обираючи метод за внутрішньою оцінкою,

дослідник обрав би найоптимістичніший, а не найкращий.

Вибір методу оптимізації за обмежених обчислювальних ресурсів. Для задач, подібних до цієї, де кілька гіперпараметрів і пласка поверхня якості, найдоцільніший **випадковий пошук з невеликим бюджетом** (20–50 точок). Він дав ту саму незміщену якість, що й повний перебір, за чверть часу. Його кандидатів можна навчати незалежно, тож він повністю використовує всі ядра. Він найменше схильний до оптимізму вибору, а бюджет можна зупинити будь-коли без втрати результату. **Байєсівська оптимізація** виправдана, коли одне навчання дороге (години, а не секунди), а гіперпараметрів багато: тоді її здатність зосередитися на кращій області економить найбільше. Але вона послідовна, гірше використовує паралельні ресурси і найсильніше «підганяє» результат під конкретні блоки, тож її оцінку обов’язково слід перевіряти вкладеною крос-валідацією або на окремій вибірці. **Повний перебір** доцільний лише для одного-двох гіперпараметрів з відомими діапазонами: його вартість зростає експоненційно з кількістю параметрів, а більшість оцінок витрачається на неважливі виміри.

Головний висновок роботи: якість звіту про модель визначає не стільки метод пошуку гіперпараметрів, скільки чесність оцінювання. Оптимізація дала лише +0,003–0,004 PR-AUC, а неправильна організація конвеєра змінила оцінку на 0,05–0,7. Тому спочатку потрібно забезпечити, щоб усі кроки, які навчаються на даних, були всередині конвеєра, і відокремити вибір гіперпараметрів від оцінювання якості (вкладена крос-валідація або окрема тестова вибірка). Лише після цього має сенс обирати метод пошуку, і за обмежених ресурсів ним має бути випадковий пошук.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

[1] Strack B., DeShazo J. P., Gennings C. et al. Impact of HbA1c Measurement on Hospital Readmission Rates: Analysis of 70,000 Clinical Database Patient Records // BioMed Research International. 2014. Vol. 2014. Article ID 781670.

[2] Pedregosa F., Varoquaux G., Gramfort A. et al. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. 2011. Vol. 12. P. 2825–2830.

[3] Akiba T., Sano S., Yanase T., Ohta T., Koyama M. Optuna: A Next-generation Hyperparameter Optimization Framework // Proc. 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2019. P. 2623–2631.

[4] Ke G., Meng Q., Finley T. et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree // Advances in Neural Information Processing Systems 30. 2017. P. 3146–3154.

[5] Saito T., Rehmsmeier M. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets // PLoS ONE. 2015. Vol. 10, No. 3. e0118432.

[6] Bergstra J., Bengio Y. Random Search for Hyper-Parameter Optimization // Journal of Machine Learning Research. 2012. Vol. 13. P. 281–305.

[7] Bergstra J., Bardenet R., Bengio Y., Kégl B. Algorithms for Hyper-Parameter Optimization // Advances in Neural Information Processing Systems 24. 2011. P. 2546–2554.

[8] Snoek J., Larochelle H., Adams R. P. Practical Bayesian Optimization of Machine Learning Algorithms // Advances in Neural Information Processing Systems 25. 2012. P. 2951–2959.

[9] Kaufman S., Rosset S., Perlich C., Stitelman O. Leakage in Data Mining: Formulation, Detection, and Avoidance // ACM Transactions on Knowledge Discovery from Data. 2012. Vol. 6, No. 4. Article 15.

[10] Micci-Barreca D. A Preprocessing Scheme for High-Cardinality Categorical Attributes in Classification and Prediction Problems // ACM SIGKDD Explorations Newsletter. 2001. Vol. 3, No. 1. P. 27–32.

[11] Chawla N. V., Bowyer K. W., Hall L. O., Kegelmeyer W. P. SMOTE: Synthetic Minority Over-sampling Technique // Journal of Artificial Intelligence Research. 2002. Vol. 16. P. 321–357.

[12] Varma S., Simon R. Bias in Error Estimation When Using Cross-Validation for Model Selection // BMC Bioinformatics. 2006. Vol. 7. Article 91.

[13] Cawley G. C., Talbot N. L. C. On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation // Journal of Machine Learning Research. 2010. Vol. 11. P. 2079–2107.